O.D.I.N.

Omni-Dimensional Intelligence Node

Product Requirements Document

Version 1.2 — Phase 2: JARVIS Interface & Knowledge Hub

Zero-Cost, Voice-Capable, Persistent-Memory Pivot + Real-Recall & Outcome-Feedback Addendum

Date: 13 August 2026 (v1.2 addendum)

Product Owner: Project Owner (solo developer)

Build tooling: Google Antigravity IDE (AI-assisted build, human-reviewed per §6)

Builds on PRD v0.2.2 (Phase 1), v1.0 (Phase 2 base), and v1.1 (Real-Recall); this addendum closes the outcome-tracking gap

Table of Contents

0. Document Control & Coverage Map

Product name: O.D.I.N. — Omni-Dimensional Intelligence Node

Version: 1.2 — Phase 2, "JARVIS Interface & Knowledge Hub," Real-Recall & Outcome-Feedback Addendum. Builds on v1.1, v1.0, and the frozen Phase 1 spec (PRD v0.2.2, 8 Aug 2026); replaces none of them. Each prior version remains the historical record of what preceded this addendum.

Date: 13 August 2026

Author / Product Owner: Project Owner (solo developer) — Accountable role throughout, unchanged from Phase 1.

Governing context: None — personal project, unchanged from v0.2.2. No accelerator, client engagement, or corporate roadmap governs this document.

Budget ceiling: $0 hard ceiling — strictly free-tier or open-source components only. Phase 2 tightens Phase 1's wording ("for as long as a free-tier path exists") into an explicit rule, because the founder interview below caught a real loophole: a component that is free-to-start but converts to usage billing after a starter credit is exhausted (e.g. Chroma Cloud's $5 credit) does not qualify as zero-cost. "Free-tier" in this document means a durable, non-depleting allowance, not a trial.

Primary consumers of this document: Unchanged from v0.2.2 — both the human developer (the Product Owner) and an AI coding agent (Google Antigravity IDE), written as an unambiguous contract wherever a requirement will be handed directly to the agent.

Why v1.0 exists

PRD v0.2.2 sketched Phase 2 in three lines, under "Future-Readiness," as a dormant bullet: pivot to a Next.js dashboard, add real-time two-way voice via the Gemini Live API, and integrate ChromaDB for persistent case history. It was never specified to build-ready detail, and it was never checked against Phase 1's own Priority #1 ("Zero-Cost, Always").

A structured founder interview on 12 August 2026 — the same gap-closure process that produced v0.2.1 from v0.2 — checked that sketch against the zero-cost rule and found two real contradictions, not just missing detail:

- The Gemini Live API bills continuously per audio token (roughly 32 input + 25 output tokens per second of audio) rather than per bounded call. Phase 1's zero-cost claim rested specifically on four calls per session sitting "trivially inside the daily free quota" — a live, open-ended voice conversation does not have that same bounded shape, and a strict $0 ceiling can't tolerate that uncertainty.
- Chroma Cloud, the natural managed home for the originally-named ChromaDB, turned out on inspection to be a $0-to-start, then usage-billed product (a one-time starter credit, not a perpetual free tier) — which fails the tightened zero-cost rule above.

Both are resolved in this document with genuinely free, durable substitutes (Sections 3, 7, 10). Per the design-freeze rule this document carries over from v0.2.2 §6, this PRD — not the original three-line sketch, and not any verbal or chat-session restatement of it — is now the frozen spec for Phase 2.

Trigger status

v0.2.2 §18 named Phase 2's trigger as "Phase 1 stable and the owner wants persistent memory." Per the founder interview, Phase 1's own success metric — five real decisions run through the complete four-persona-plus-Judge pipeline (v0.2.2 §3) — has been reached. Phase 2 is proceeding on a validated basis, not speculatively.

v0.2.2 → v1.0 (Phase 2) — what carried over, what changed

Area	v0.2.2 (Phase 1, frozen)	v1.0 (Phase 2)
Reasoning engine	Four-persona-plus-Judge, Gemini Flash	Unchanged, byte-for-byte (§20 reproduced in full below). Only the pinned model identifier is updated.
Frontend / host	Streamlit on Streamlit Community Cloud	Next.js (React) on Vercel's free tier
Voice	None (typed fields only)	Two-way voice I/O via the browser's native Web Speech API — explicitly not the Gemini Live API
Persistence	None — fully stateless between sessions	Supabase (Postgres + pgvector) — persistent, RLS-protected case history; sensitive narrative field encrypted at the app layer
Backend process model	Streamlit's persistent websocket process	Next.js serverless API routes — viable precisely because voice no longer requires a long-lived connection
Migration	N/A	Hard cutover: the Streamlit app is retired the moment Phase 2 build starts; no parallel run (§21.4)
Cognitive engine model	"Current-gen Gemini Flash," confirm exact GA model at build time	Confirmed: Gemini 3.5 Flash, per the founder interview, 12 Aug 2026
Data-at-rest posture	None — the public repo held code only, never user content	Real user content now persists server-side for the first time — §8 is rewritten around this fact, not just amended
Content-hygiene nudge	Should-have, soft reminder (v0.2.2 FR-9)	Same soft-reminder posture retained — but now backed by real technical controls (RLS + encryption + server-side-only access), not the only safeguard in the system

Why v1.1 exists

v1.0 scoped the Supabase knowledge hub as a passive archive: sessions were written after completion, but nothing was ever read back in. On review, that fell short of what §3 actually named as the Phase 2 problem — "no case history, no pattern-tracking across past decisions." An archive gives you the history;

it doesn't give you the pattern-tracking. v1.1 closes that gap by promoting real cross-session retrieval from Could-Have to Must-Have (§5), and answers a second question raised in the same review: whether O.D.I.N.'s memory should integrate with a separate, not-yet-built personal knowledge-hub project (Perplexity/Gemini/Claude + NotebookLM + Obsidian, still just an idea with no architecture decided). The decision, recorded here rather than left implicit: build the real recall feature inside O.D.I.N. now, on a substrate boring and portable enough (plain Postgres, documented export) that it can feed into that project later without a rewrite — but do not couple O.D.I.N.'s shipped feature to an undated external dependency. Full integration remains explicitly deferred (§18) until that project has its own spec.

v1.0 → v1.1 — what changed

Area	v1.0	v1.1
Cross-session recall	Could-Have, deferred — sessions stored but never read back in	Must-Have — the Judge call now receives similarity-matched past syntheses before it arbitrates (§5, §10, §12)
Judge input/output	Three persona envelopes only (§20.7, unchanged since v0.2.2)	Three persona envelopes plus an optional past_context array; Judge output gains an optional pattern_note field — the one deliberate, versioned amendment to the otherwise-frozen §20
Data portability	Implicit (plain Postgres)	Explicit requirement: a documented JSON export path and stored embedding-model versioning, so history can migrate into a future system without a rebuild (§10, §12)
Relationship to the separate personal-brain project	Not addressed	Explicitly deferred, not abandoned — recorded as an open item with its own rationale (§18, §22), not silently assumed either way

Coverage map

This PRD keeps the same section skeleton v0.2.2 established, for continuity across phases. The same two sections remain marked not applicable rather than silently omitted:

- Section 2 (Program / Stakeholder Alignment) — still not applicable; no external program governs this project.
- Section 9 (API Specification) — still not applicable; ODIN's Next.js API routes are internal implementation, not a surface exposed to external callers.

Section 20 (Persona Prompt Specification & Output Schema) carries the Quant, Strategist, and Behaviorist prompts unmodified from v0.2.2, per the design-freeze rule. The Judge alone is amended — now twice, in v1.1 and again in v1.2 — with each change isolated and explained rather than blended in silently, exactly as §20.8 requires. Section 21 remains repurposed from v0.2.2's UI-copy-and-scenarios

section into voice UI copy and the migration runbook, now also carrying the outcome-prompt copy strings (§21.2).

Why v1.2 exists

v1.1 made the knowledge hub genuinely active: the Judge could recognize a pattern in a past session's reasoning. What it still couldn't do was know whether that past reasoning was actually right — recall surfaced what O.D.I.N. once concluded, never what happened afterward. On review, that's the difference between a decision log and a brain: a log remembers what was said; something that learns has to know what worked. v1.2 closes that gap with a deliberately lightweight, low-friction outcome-feedback loop, layered entirely on top of what v1.1 already built — no new provider, no new architecture, no reopening of the Quant/Strategist/Behaviorist independence that's been frozen since v0.2.2.

v1.1 → v1.2 — what changed

Area	v1.1	v1.2
Outcome tracking	None — the Judge saw past reasoning only, never whether it was validated	New session_outcomes table; the Judge's past_context now carries a status and optional narrative_summary for any past session with a recorded outcome (§10, §20.7)
Capture UX	N/A	Primary: opportunistic prompt when a new session closely matches a past one with no recorded outcome. Fallback: manual entry from the case-history detail view (§5, §12, §21.2)
Judge prompt	v1.1 amendment (past_context, pattern_note)	v1.2 amendment: explicit instruction to weigh a validated outcome more heavily than an untested past synthesis, and never to treat a missing outcome as a negative signal (§20.7)
Data model	sessions table only	+ session_outcomes table (1:1 with sessions), same RLS and app-layer-encryption treatment as raw_narrative (§8, §10)
Export (FR-25)	Session, persona, and Judge data	Now also includes any recorded session_outcomes rows, encrypted-narrative-by-default, unchanged posture (§10, §12)
Persona independence	Judge-only memory, by design	Unchanged — outcome data stays Judge-only in this version; feeding validated outcomes to the personas is

Area	v1.1	v1.2
		named as a distinct, deferred decision, not folded in here (§18)

1. Executive Summary

Phase 2 pivots O.D.I.N.'s delivery mechanism — from a Streamlit MVP to a persistent, voice-capable web app — without touching the one component the project has explicitly decided not to touch: the four-persona-plus-Judge reasoning core. Per Guiding Priority #3 ("Architecture Is Disposable, the Reasoning Design Isn't," §3), the frontend, the host, and even the interaction mode (typed vs. spoken) are all free to change. Section 20's prompts and JSON schemas are carried into this document unmodified except for the Judge, which is amended — deliberately, twice, each time through this same versioned document rather than mid-build.

The two headline technical ideas in the originally-pitched Phase 2 concept — real-time voice via Gemini's Live API, and a ChromaDB Cloud-hosted knowledge hub — did not survive contact with Priority #1 ("Zero-Cost, Always") once actually priced out. Live API bills continuously per audio token rather than per bounded call; Chroma Cloud's "free tier" is a one-time starter credit followed by usage billing, not a durable allowance. Both are replaced in this spec with substitutes that are genuinely, durably free: the browser's own Web Speech API for voice I/O (no Gemini cost at all, no separate service), and Supabase's real free-tier Postgres with the pgvector extension for persistent, semantically-searchable case history.

Target: 4–6 weeks from build start (versus Phase 1's 1–2 weeks) — reflecting a genuinely larger surface area (a full frontend rewrite, a new persistence layer, a new interaction mode), while the reasoning core itself needs no rework at all.

v1.1 made the knowledge hub genuinely active rather than a write-only archive: before the Judge arbitrates, a similarity search over past sessions surfaces recognizable patterns, which the Judge may note in its synthesis. v1.2 adds the piece that makes that recall trustworthy rather than merely present: a lightweight outcome-feedback loop, captured with minimal friction and fed back into the exact same retrieval path, so the Judge can eventually tell the difference between a past conclusion that held up and one that didn't. Target for this addendum: 1–2 weeks after v1.1's own milestones sign off — additive, not a rearchitecture, and deliberately not stacked into v1.1's own build window (§11).

As with every prior version, this document is written to be sufficient on its own: everything Antigravity needs to build Phase 2 through this addendum, AI-assisted and human-reviewed per §6, is contained here.

2. Program / Stakeholder Alignment

Not applicable, unchanged from v0.2.2. No accelerator program, internal corporate roadmap, or client engagement governs this document. The only external constraints are the free-tier terms and rate limits of Vercel, Supabase, and Google AI Studio — tracked in Section 7, not here.

3. Vision & Problem Statement

The problem (Phase 2)

Phase 1 proved the reasoning wedge works: five real decisions were run through the complete four-persona-plus-Judge pipeline and produced structured, adversarially-tested breakdowns that a single generic LLM chat session doesn't produce unprompted. What Phase 1 does not do is remember anything. Every session starts from zero — no case history, no pattern-tracking across past decisions, and no lower-friction way to use it than typing three text fields on a phone, sometimes exactly when typing itself is the barrier to using the tool in the moment a decision is actually live.

Definition of success — Phase 2 (proposed, confirm before build)

By the end of Phase 2 (target 4–6 weeks from build start), the Product Owner can: (a) run a complete four-persona-plus-Judge session by voice from a phone browser as easily as by typing; (b) see and re-open at least one past decision's full case history from a later session; and (c) confirm the $0 hard ceiling held throughout — no paid tier triggered anywhere in the stack, across Vercel, Supabase, and the Gemini API. This mirrors how v0.2.2 §3 stated a success definition; unlike that one, this wording was proposed during drafting rather than confirmed word-for-word in the interview — treat it as a default to correct, not a settled fact (see §22 open questions).

Competitive landscape & wedge

Unchanged from v0.2.2: generic chat LLMs converge to a single hedged answer; decision-journaling apps track decisions with no reasoning engine behind them. Phase 2 sharpens O.D.I.N.'s wedge by adding memory — a capability journaling apps have and reasoning engines don't, and vice versa for generic chat LLMs. No competitor in this landscape combines forced adversarial reasoning with persistent case history at zero cost.

Positioning statement (binding, updated)

“O.D.I.N. is a zero-cost decision-support system that forces high-stakes dilemmas through four independent reasoning passes — cold optimization, adversarial strategy, behavioral-bias audit, and first-principles arbitration — rather than returning one generalized, risk-averse answer. It is now accessible by voice and builds a persistent case history across decisions, without compromising the zero-cost or reasoning-design commitments that defined Phase 1.” Keep this sentence consistent across all future materials describing the project.

Guiding priorities, in order — carried over, one clause tightened

- **1. Zero-Cost, Always —** tightened for Phase 2: not just “$0 spend today,” but “no component whose free access is a depleting credit rather than a durable tier.” This is the rule Chroma Cloud failed and Supabase passed (§0, §7).
- **2. Feature-Complete Over Fast —** Phase 2 ships voice I/O and persistent memory together, not staggered — the founder's explicit choice against deferring voice to a later sub-phase.
- **3. Architecture Is Disposable, the Reasoning Design Isn't —** this is the entire thesis of Phase 2. Frontend, host, and interaction mode all change; Section 20 does not.
- **4. Privacy Is Knowingly Traded Away —** revised in substance, not just carried forward — see Section 8. The repo stays public, but for the first time genuine user content persists server-side, so this trade-off now needs technical backing (RLS, encryption, server-side-only access), not disclosure alone.

4. Stakeholders & Actor Separation (RACI)

Unchanged from v0.2.2. Single human stakeholder: the Product Owner is simultaneously the paying "customer" (of their own $0 budget), the day-to-day user, and the sole beneficiary. No separate paying customer, no licensed-professional sign-off requirement, and no regulator in the loop — O.D.I.N. remains a personal reasoning tool, not a service offered to others (see §19 for the resulting advisory-disclaimer requirement).

Lifecycle task	Responsible	Accountable	Consulted	Informed
PRD / architecture spec	Architect LLM (drafting assist)	Product Owner	Uploaded governance protocols; frozen v0.2.2 spec	—
Code implementation	Antigravity coding agent	Product Owner	This PRD (frozen spec)	—
Guardrail / hook enforcement	Antigravity deterministic hooks (automated)	Product Owner	—	—
Test scaffolding & execution	Antigravity coding agent	Product Owner	Section 14 gates	—
Deployment to Vercel	Product Owner (manual push / connected repo)	Product Owner	—	—
Streamlit decommission (§21.4)	Product Owner	Product Owner	—	—

Per the AIM framework's non-negotiable constraint, carried over unchanged: no autonomous subagent is ever assigned Accountable status. The Product Owner is Accountable for every row above, with no exception.

5. MVP Scope (MoSCoW) — Phase 2

Must-Have

- Full frontend rewrite: a Next.js (React) app replacing Streamlit entirely, deployed on Vercel's free tier.
- Server-side-only architecture: every Gemini and Supabase call happens inside Next.js API routes; no API key or database credential ever reaches client-side JavaScript.
- Two-way voice I/O via the browser's native Web Speech API — SpeechRecognition for input, SpeechSynthesis for output. Gemini is never used for voice; it remains reasoning-only, exactly as in Phase 1.
- Model string updated in its one config variable to Gemini 3.5 Flash (or whatever confirmed GA model is current at build time — same confirm-at-build-time posture v0.2.2 used).

- Section 20's four persona prompts and JSON schemas carried over byte-for-byte, unmodified, into the new codebase.
- Supabase (Postgres + pgvector) persistent case-history store: every completed session's structured intake, all four persona/Judge JSON outputs, and a summary embedding are written after the session completes.
- Row-Level Security enabled on every Supabase table from the first migration — not deferred until a hypothetical future multi-user version.
- App-layer encryption (Node's built-in crypto module) on the raw_narrative field specifically, before it's written to Supabase; decrypted only server-side, only on read.
- A minimal case-history view: past sessions are listable and individually re-openable.
- Real cross-session recall (new in v1.1, promoted from Could-Have): before the Judge call, a similarity search over past sessions' embeddings surfaces the 2–3 most relevant prior syntheses, passed to the Judge only — never to the three independent personas — as optional past_context. The Judge may surface a recognized pattern in its synthesis and tension_points, and in a new optional pattern_note field (§20.7). This is what makes the knowledge hub an active part of the reasoning process rather than a passive log.
- Data portability (new in v1.1): a documented JSON export — one session, or all sessions — in a format independent of Supabase's internal representation, plus the embedding-model name stored alongside every vector. This is the seam for migrating into a future, separately-scoped personal knowledge-hub project without a rebuild (§18).
- Outcome-feedback capture (new in v1.2): a session_outcomes table records, for any past session, a required status (followed_path / deviated / still_deciding) and an optional narrative reflection. Primary capture path is opportunistic — when a new session closely matches a past one with no recorded outcome, the app asks before the new session begins, always skippable in one tap (§10, §12, §21.2).
- Outcome-aware recall (new in v1.2): the Judge's past_context entries now include the outcome status and narrative summary when one was recorded, with explicit prompt instruction to weigh a validated outcome more heavily than an untested past synthesis, and to never treat a missing outcome as a negative signal (§20.7).
- JARVIS-aesthetic UI redesign: a modern, animated, dark-themed dashboard replacing Streamlit's default look, built with CSS / Framer Motion — no paid design tooling.
- Hard cutover from Streamlit: the Phase 1 app is retired the moment Phase 2 build starts. This PRD records that decision; it is not a prompt to revisit it mid-build.
- $0 spend verified across Vercel, Supabase, and the Gemini API at every milestone close — the same rigor v0.2.2 applied to its own zero-cost adherence checks.

Should-Have

- Content-hygiene nudge carried over from Phase 1 (a soft reminder, not a hard block) — paired with, not replacing, the technical controls above.
- Graceful handling of Supabase's free-tier auto-pause (projects pause after 7 days of inactivity): a clear “reconnecting” state, never a silent failure or a raw exception.
- Graceful voice-unavailable fallback: on a browser without Web Speech API support, the app degrades cleanly to Phase 1's typed-field flow.

- A session delete/purge capability for the owner's own stored sessions. This was not explicitly confirmed in the founder interview; it's added here as a reasonable inference from the fact that real content now persists indefinitely for the first time — flagged in §22 as an item to confirm before Milestone 3, not a settled requirement.
- If recall (above) risks the 4–6 week target, ship it as a fast-follow Milestone 7 rather than blocking the rest of Phase 2's sign-off (§11).
- Manual outcome entry (new in v1.2): a past session's detail view lets the owner add or edit its outcome directly, independent of whether the opportunistic prompt ever fired — the fallback path for sessions that never get a similar successor.

Could-Have (deferred, not built this version)

- Deep integration with the separate personal knowledge-hub project (Perplexity/Gemini/Claude + NotebookLM + Obsidian) — deferred, not rejected, until that project has its own defined architecture. See §18 for the reasoning.
- Any multi-user auth (e.g. Supabase Auth) — still exactly one user; would only matter if RLS needed to actually differentiate rows between people.
- Distinct synthesized voices per persona — a cosmetic layer, not core to the pivot's purpose.
- Feeding validated outcome data to the Quant, Strategist, or Behaviorist (new in v1.2, deferred): outcomes are ground truth rather than opinion, so this is a different and arguably safer proposition than feeding them past reasoning — but it still touches the persona-independence design principle frozen since v0.2.2, and deserves its own explicit decision rather than riding in on this addendum (§18).

Won't-Have (explicitly out of scope)

- The Gemini Live API, in any form — explicitly rejected in the founder interview on cost-predictability grounds, not merely deferred.
- Chroma Cloud, or any other starter-credit-then-billed service — the rejection generalizes beyond Chroma specifically to the whole product category.
- A stateful multi-turn agentic loop, a LangGraph rebuild, or a Python interpreter tool — still Phase 3 (§18).
- Any paid tier anywhere in the stack.
- Parallel-running the old Streamlit app as a fallback during the build — explicitly rejected per the hard-cutover migration decision (§21.4).
- A numeric outcome rating scale (new in v1.2, rejected not deferred): a 1–5 score invites noise the Judge can't act on. Structured status plus optional free text was chosen deliberately over this (§0 rationale, recorded in this conversation's own review).

Hard target: Phase 2's 4–6 week window is a self-imposed internal target for v1.0/v1.1's scope; v1.2 is scoped separately at 1–2 weeks after that window closes, not folded into it (§11, §19).

6. Development Governance & Agentic Architecture

Design-freeze rule, carried over unchanged: once this PRD is saved as the project's frozen spec, any requirement change must be written into a new PRD version before it is touched in code. A change

typed directly into an Antigravity chat session mid-build is not a valid source of new scope — this document, v1.1, is itself an example of the rule working as intended: the recall feature was raised in conversation, but nothing changes in code until it's written here first. v1.1 is now the frozen spec for Phase 2.

Layer 1 — Memory / Constitution

- Never hardcode the Gemini API key, the Supabase service-role key, or any other secret — only Vercel's environment-variable store. This extends v0.2.2's single-secret rule to a second secret; the rule itself doesn't change.
- Never expose the Supabase service-role key, or any key capable of bypassing RLS, to client-side code — no exception, including for debugging convenience.
- Never commit to main without human diff review; no "accept all," unchanged from Phase 1.
- Never auto-run terminal commands without human-in-the-loop confirmation — auto-run stays OFF, non-negotiable, unchanged.
- Never write real third-party personal data into seed or test files — unchanged, and now more consequential given Supabase actually persists content.
- Never auto-render remote images or URLs in chat — unchanged, disclosed credential-exfiltration channel.

Layer 2 — Knowledge / Skills

- git_workflow.md — carried over unchanged (§16).
- gemini_api_integration.md — carried over, updated for the Gemini 3.5 Flash model string and current free-tier limits.
- persona_prompts.md — carried over byte-for-byte from v0.2.2; the coding agent implements these as given, unmodified (§20).
- supabase_integration.md (new) — documents the RLS policy definitions, the pgvector schema, the app-layer encryption/decryption code path, and the free-tier auto-pause behavior to design around.
- web_speech_api.md (new) — documents SpeechRecognition/SpeechSynthesis browser support, the typed-field fallback trigger condition, and language/locale handling.

Layer 3 — Guardrails / Hooks

- In-session: human confirmation required before any terminal action beyond read-only (auto-run disabled) — unchanged.
- Commit-time: pre-commit secret scan (gitleaks) required before every commit — unchanged, now scanning for two secret patterns instead of one.
- Deploy-time: manual push to Vercel (or a connected-repo auto-deploy the human explicitly triggers) — the human remains the deployment gate; no CI/CD pipeline is warranted at this scale.

Layer 4 — Delegation / Subagents

Not warranted at this scale, unchanged from v0.2.2. One Antigravity session, one scope — building v1.0's Next.js app — rather than a supervisor/subagent swarm.

Layer 5 — Distribution / MCP & external connectors

- Adopt now: none. Phase 2 only needs the Gemini API and the Supabase client library; no MCP server is required.
- Defer: a GitHub MCP integration for automated PR management — still disproportionate for a solo repo.
- Out of scope: any connector with reach into production credentials or external communication tools — unchanged.

7. Tech Stack & System Architecture

Component	Selected technology	Rationale
Frontend	Next.js (React), deployed on Vercel's free tier	Replaces Streamlit per Priority #3. Native serverless API routes remove the need for a separately-hosted backend.
Voice I/O	Browser-native Web Speech API (SpeechRecognition + SpeechSynthesis)	Zero cost, zero token billing, no separate service to provision. Chosen over the Gemini Live API after pricing verification found continuous per-audio-token billing incompatible with a strict $0 ceiling (§0).
Backend	Next.js serverless API routes — same deployment as the frontend, no separate server	Removing the WebSocket requirement (voice is now handled entirely client-side by the browser) means the reasoning engine's bounded request/response calls fit serverless cleanly, unlike a real-time voice stream would have.
Persistent storage	Supabase (Postgres + pgvector), free tier	A genuine durable free allowance (500MB), not a depleting starter credit like Chroma Cloud. pgvector provides semantic search without a separate vector-database vendor.
Cognitive engine	Gemini 3.5 Flash, model string pinned in one config variable	Confirmed current model per the founder interview, 12 Aug 2026. Free-tier access is retained for Flash-class models as of this writing; confirm exact current rate limits in Google AI Studio at build time — same posture v0.2.2 applied to its own model choice.
Output mode	Gemini structured output (JSON mode / responseSchema)	Unchanged from v0.2.2 — constrains every persona and Judge response to the Section 20 schemas at the API level.
Repository	GitHub (public)	Unchanged posture. Secrets management now covers two keys — the Gemini API key and the Supabase service-role key — both as Vercel

Component	Selected technology	Rationale
		environment variables, neither ever committed to the repo.

Third-party dependencies

Google Gemini API (free tier), Supabase (free tier), Vercel (free tier). No others. Each is checked against the tightened zero-cost rule in §0 — durable free allowance, not a depleting credit.

System flow

The user speaks or types structured input in the Next.js UI; if spoken, the Web Speech API transcribes it to text entirely client-side, with no Gemini involvement. A Next.js API route then sequentially calls Gemini three times for Quant, Strategist, and Behaviorist, then once for the Judge — the identical flow Section 20 defines. On completion, the full session (structured intake plus all four JSON outputs) is written to Supabase, with the raw_narrative field encrypted before write and every row protected by Row-Level Security. The multi-tab report renders in the new UI and can optionally be read aloud via SpeechSynthesis. The session then appears in a case-history list for later reference.

What stays the same as this scales

Identical framing to v0.2.2: the reasoning design's stability was already established in Phase 1; Phase 2 exercises that stability rather than re-testing it. If a future paid or scaled tier is ever built, Section 20's JSON schemas — and now the Supabase schema built on top of them (§10) — are both designed to extend rather than require a rewrite.

8. Cybersecurity & Compliance Framework

Calibration — rewritten, not merely amended

v0.2.2's entire security posture rested on one load-bearing fact, stated plainly in its own §8: “no database exists in Phase 1, so there is no server-side data-at-rest to protect... the only persistent artifact is the public repo, which holds code only, never user content.” Phase 2 breaks that fact. For the first time, O.D.I.N. is a controller of real, persistent personal narrative content — including, per Scenario A (§21.3), content that may reference real third parties. This remains a single-user personal tool with no formal data-processing relationship to any outside party, but this section is rewritten around the new fact, not just amended to mention it.

Adopted now

- Secrets: two secrets now exist — the Gemini API key and the Supabase service-role key. Both live only in Vercel's environment-variable store; neither is ever committed to the repo or pasted into an Antigravity chat session. Pre-commit secret scan (gitleaks) before every commit, scanning for both patterns.
- Server-side-only data access: no Supabase key of any kind is included in any client-side bundle. All reads and writes happen inside Next.js API routes, never in the browser.

- Row-Level Security enabled on every Supabase table from the first migration — defense-in-depth that doesn't depend on a future auth layer being added; it protects the table today even with one user.
- App-layer encryption on the raw_narrative field, using a symmetric key stored as a Vercel environment variable and Node's built-in crypto module (no paid key-management service). Encrypted before write; decrypted only server-side, only on read. Other fields — the persona/Judge JSON outputs and the structured intake's objective/constraint text — are not encrypted by default in this version, since they carry materially lower re-identification risk than free-text narrative content. This judgment is flagged as revisitable, not final (§22).
- Same encryption treatment extended to session_outcomes.narrative (new in v1.2): an outcome reflection can carry exactly the same kind of sensitive, third-party-referencing content as raw_narrative — "it didn't work out because she said no" is not meaningfully less sensitive than the original narrative — so it gets the identical symmetric-key encryption, not a lighter default. The status field (followed_path / deviated / still_deciding) is low-risk structured metadata and stays plaintext, matching the objective/constraint reasoning above.
- Content-hygiene nudge — carried over as a soft reminder (continuing v0.2.2 FR-9's intent), now backed by, not substituting for, the technical controls above.
- All Phase-1 Antigravity-specific controls carried over unchanged: auto-run off; remote-content auto-rendering disabled; dependency hygiene checks before any new package is accepted; indirect prompt injection from fetched content treated as untrusted and never acted on without flagging to the human; every AI-generated diff human-reviewed before merge; no auto-merge to main, ever.
- Data minimization, reframed: no third-party data-processor relationship is created by using Supabase or Vercel — both process data as infrastructure providers under their own standard terms, not as parties O.D.I.N. has a bespoke agreement with. This remains a personal-tool posture, not an enterprise one, even though real content now persists.

New risk, explicitly disclosed rather than assumed away

Supabase's and Vercel's own account-level security now becomes a dependency for the first time. If either account were ever compromised, encryption protects raw_narrative and, as of v1.2, session_outcomes.narrative — but the persona/Judge JSON outputs and the structured intake's objective/constraint fields would be exposed as plaintext. This is an accepted risk at this scale, named here on purpose, consistent with how v0.2.2 named its own public-repo trade-off rather than hiding it (§1). It is a narrower and more consequential risk than Phase 1's public-repo trade-off, because it now concerns real content rather than only code.

Explicitly deferred or not applicable at this scale

Control class	Why deferred / N/A now	Where it moves if this scales
MFA, SSO, RBAC/ABAC, passwordless, SCIM	No auth system exists — single user, no login at all, unchanged from Phase 1	Only if multi-user access is ever built
Service-mesh microsegmentation, Identity-Aware Proxies	One Next.js app on one host; no microservices	Not applicable unless the architecture changes fundamentally

Control class	Why deferred / N/A now	Where it moves if this scales
Payment handling (tokenization, PCI scope)	No payment flow exists anywhere in this project	N/A unless O.D.I.N. is ever monetized directly
ML-driven DLP, WORM audit logs, AI SBOMs, formal Policy-as-Code / GRC auditing	Enterprise-scale controls, disproportionate for a zero-cost solo app	Future Roadmap (§18) only if productized
BASIC/A2AS framework, OVON multi-agent defense pipeline	No autonomous multi-agent loop exists by design — unchanged	Revisit at Phase 3 if a LangGraph rebuild introduces genuine autonomy
Formal GDPR/PDPA/HIPAA compliance mapping	No controller/processor relationship with a third party is currently triggered	Open item — verify with a lawyer before any expansion beyond a single owner, and sooner than Phase 1 needed to, given real content now persists

9. API Specification

Not applicable, unchanged from v0.2.2. O.D.I.N. consumes the Gemini API and the Supabase client library as a client (§7); its own Next.js API routes are internal implementation detail, not an API surface exposed to external callers in v1.0.

10. Data Model

Structured intake and the persona/Judge JSON envelopes are unchanged, byte-for-byte, from v0.2.2 §10 and §20.3–20.7. What's new in Phase 2 is where that data goes after a session completes.

Structured intake (unchanged)

```
{
  "core_objectives": string,
  "known_constraints": string,
  "raw_narrative": string
}
```

Supabase schema — sessions table (new)

Column	Type	Notes
id	uuid (pk)	Generated server-side on write.
created_at	timestamptz	Default now().
core_objectives	text	Plaintext — lower sensitivity than raw_narrative.
known_constraints	text	Plaintext — lower sensitivity than raw_narrative.

Column	Type	Notes
raw_narrative_encrypted	bytea	App-layer ciphertext; encrypted before write, decrypted only server-side on read (§8).
quant_output	jsonb	Full Quant envelope, §20.4.
strategist_output	jsonb	Full Strategist envelope, §20.5.
behaviorist_output	jsonb	Full Behaviorist envelope, §20.6.
judge_output	jsonb	Full Judge output, §20.7.
narrative_embedding	vector (pgvector)	Computed from the Judge's synthesis text, not the raw narrative directly — deliberately, so the embedding itself can't become a shortcut around the encryption on raw_narrative. See implementation note below.
embedding_model	text (new, v1.1)	Name/version of the embedding model used for this row — e.g. a specific Gemini embedding model identifier. Written alongside every embedding so a future model switch is a documented, detectable re-embed step, not a silent incompatibility.

Implementation note for Antigravity: do not embed raw_narrative directly, even transiently in memory, into narrative_embedding's source text. Use the Judge's synthesis field, which is already a downstream abstraction and was never the raw sensitive input. This preserves the point of encrypting raw_narrative in the first place — an embedding computed from the plaintext narrative would otherwise leak much of what the encryption was meant to protect.

Retrieval flow (v1.1, extended in v1.2)

Before the Judge call fires, the app embeds the current session's core_objectives and known_constraints using the same embedding step already used for narrative_embedding, then queries Supabase with pgvector's cosine-similarity operator for the 2–3 most similar past sessions. Only each match's judge_output.synthesis is retrieved — never raw_narrative_encrypted — preserving the same leak-avoidance logic used for the embedding itself. As of v1.2, if a matched session has a row in session_outcomes, its status and (if present) narrative are retrieved too and folded into that match's entry. The result is passed to the Judge call as an optional past_context array; on a person's first few sessions, this array is simply empty, and the Judge behaves exactly as it did in v1.0.

Supabase schema — session_outcomes table (new in v1.2)

Column	Type	Notes
id	uuid (pk)	Generated server-side on write.
session_id	uuid (fk → sessions.id)	One outcome row per session, at most — enforced with a unique constraint, not just convention.

Column	Type	Notes
status	text	One of: followed_path, deviated, still_deciding. Required — there is no outcome row without a status (FR-28).
narrative_encrypted	bytea	App-layer ciphertext, same treatment as sessions.raw_narrative_encrypted (§8). Optional — a status alone is a complete, valid outcome record.
prompted_via	text	opportunistic or manual — records how the outcome was captured, kept for later UX tuning (e.g. measuring how often the opportunistic prompt actually gets used).
recorded_at	timestamptz	Default now().

Implementation note for Antigravity: session_outcomes is intentionally a separate table, not new columns on sessions. Sessions rows are written once, on completion, and are otherwise treated as an append-only record of what the reasoning engine produced; outcomes are written later, sometimes much later, and sometimes edited (FR-30). Keeping them separate means the original session record is never mutated after the fact.

Row-Level Security

Enabled on both the sessions and session_outcomes tables from their respective first migrations. Even with exactly one owner, the policy is scoped to a known static owner identifier rather than left fully open — so neither table is world-readable through Supabase's REST endpoint if a key were ever mishandled or a policy misconfigured elsewhere.

Judge input and output (amended in v1.1, extended again in v1.2 — see §20.7)

The Judge still receives the three completed persona envelopes, verbatim, never the raw intake fields again — that part is unchanged since v0.2.2. What v1.1 added was the optional past_context array and a corresponding optional pattern_note field. v1.2 doesn't add a new top-level field — it enriches each past_context entry with outcome data when available. Full schema and system-prompt diff in §20.7.

Export capability (new in v1.1, extended in v1.2)

A documented function or route exporting one session, or all sessions, as plain JSON — core_objectives, known_constraints, all four persona/Judge outputs, and metadata (created_at, embedding_model). As of v1.2, any corresponding session_outcomes row is included in the same export. Both raw_narrative and session_outcomes.narrative are exported only in their encrypted form, unless the owner explicitly authenticates a decrypt-for-export step; this keeps the export capability from becoming a bypass around §8's encryption control. This JSON shape, not Supabase's internal schema, is the intended interoperability seam for a future personal knowledge-hub project (§18).

Final report shape (extended)

Same tabs structure as v0.2.2, now additionally sourced from Supabase when a past session is re-opened, rather than only freshly computed within a single session, now optionally showing the Judge's pattern_note when present, and now showing a session's recorded outcome (or an “add outcome” affordance if none exists yet) in its detail view.

11. Development Roadmap

Target: 4–6 weeks from build start, confirmed in the founder interview (§0) — wider than Phase 1's 1–2 weeks, reflecting the larger surface area.

Milestone	Scope
M1	Next.js scaffold deployed on Vercel; both environment variables (Gemini + Supabase keys) configured; gitleaks pre-commit hook carried over.
M2	Four-call sequential engine ported to Next.js API routes; Section 20 prompts and schemas copied verbatim; Gemini 3.5 Flash pinned in its one config variable.
M3	Supabase schema migrated with RLS enabled from the start; app-layer encryption implemented and verified round-trip before any real narrative content is stored; session delete capability resolved per §22 and built if confirmed.
M4	Web Speech API voice I/O wired to the intake flow and to report readback; typed-input fallback verified on a browser without Web Speech support.
M5	JARVIS-aesthetic UI build: case-history list and detail views, progressive-disclosure status strings (§21.2), phone-browser verification.
M5.5 (new, v1.1)	Cross-session recall built: embedding-then-similarity-query step wired in ahead of the Judge call; Judge prompt and schema amended per §20.7; export endpoint (one session / all sessions as JSON) implemented and verified.
M6	Section 14 security- and schema-verification pass completed, including the new recall and export paths; Streamlit app retired per the §21.4 runbook; $0 spend confirmed across Vercel, Supabase, and Gemini before Phase 2 sign-off.
M7 (conditional)	Only triggered if M5.5 threatens the 4–6 week target: recall and export ship as a fast-follow after the rest of Phase 2 signs off, per the Should-Have fallback in §5. Not a planned milestone — a named escape hatch.

v1.2 addendum — separate build window

Target: 1–2 weeks after v1.1's own milestones (through M6, or M7 if triggered) sign off. Deliberately not stacked into the same window — this is additive work on top of a proven recall feature, not a reason to widen Phase 2's own target.

Milestone	Scope
N1	session_outcomes schema migrated with RLS and app-layer encryption on narrative; round-trip verified before any real outcome content is stored.
N2	Opportunistic prompt wired into the new-session flow, reusing the existing similarity search; manual outcome entry/edit added to the case-history detail view (FR-30) as the fallback path.

Milestone	Scope
N3	Judge retrieval payload extended with outcome status/narrative_summary; §20.7's v1.2 Judge-prompt amendment implemented; pattern_note may now reference outcomes.
N4	Section 14 QA extended to cover the outcome flow end-to-end; $0 spend re-confirmed (no new provider introduced, so this is a quick re-check, not new verification work).

12. Functional Requirements

Numbering continues from v0.2.2's FR-1 through FR-11, which are unchanged and carried over by reference (the reasoning engine, structured intake, and progressive-disclosure requirements still apply, now against the Next.js implementation instead of Streamlit).

ID	Requirement	Acceptance criterion	Pri.
FR-12	Server-side-only secrets	Neither the Gemini key nor the Supabase service-role key appears in any client-side bundle, network request, or page source — verified via browser devtools at each milestone.	Must
FR-13	Voice I/O	SpeechRecognition correctly transcribes spoken intake into the same three structured fields Phase 1 used; SpeechSynthesis reads back at least the Judge's synthesis and next_3_actions.	Must
FR-14	Voice fallback	On a browser without Web Speech API support, the typed-field flow works with no console errors and no broken UI.	Should
FR-15	Persistent storage round-trip	A completed session is retrievable, byte-for-byte except the encrypted field, from a fresh page load after all browser storage is cleared.	Must
FR-16	RLS enforcement	A request made with only the public/anon Supabase key, without the service-role key, cannot read or write sessions rows.	Must
FR-17	Encryption round-trip	raw_narrative is unreadable as plaintext via direct Supabase table inspection; it decrypts correctly only through the app's server-side code path.	Must
FR-18	Reasoning-engine parity	Quant, Strategist, and Behaviorist prompts, diffed against v0.2.2 §20, are byte-for-byte identical. The Judge prompt carries two intentional amendments (v1.1: past_context/pattern_note; v1.2: outcome-aware past_context) — diffed against the prior version each time, the change is limited to what §20.7 documents, nothing else.	Must

ID	Requirement	Acceptance criterion	Pri.
FR-19	Zero-cost adherence	No paid tier is triggered on Vercel, Supabase, or Google AI Studio at any milestone close, verified against each provider's usage dashboard, not assumed.	Must
FR-20	Hard cutover	The Streamlit app is undeployed or archived — not left publicly live — once Milestone 6 passes.	Must
FR-21	Session delete	The owner can delete a stored session; the corresponding Supabase row and its embedding are removed, not merely hidden from the UI.	Should
FR-22 (new, v1.1)	Recall retrieval	Before every Judge call, a similarity query runs against past sessions' narrative_embedding; on a person's first sessions (empty history) it returns zero matches and the Judge proceeds exactly as in v1.0, with no error.	Must
FR-23 (new, v1.1)	Persona independence preserved	past_context is never present in the Quant, Strategist, or Behaviorist API payloads — verified by inspecting each of the three requests directly, not just the Judge's.	Must
FR-24 (new, v1.1)	Recall auditability	When the Judge's past_context was non-empty, pattern_note is either a specific reference to what was recognized or explicitly null — never silently omitted from the response.	Must
FR-25 (new, v1.1)	Data export	A session, or the full session history, can be exported as JSON independent of Supabase's internal representation, including the embedding_model value for every row.	Must
FR-26 (new, v1.2)	Outcome schema	session_outcomes exists with RLS enabled and a unique constraint on session_id; narrative round-trips correctly through encryption, same test pattern as FR-17.	Must
FR-27 (new, v1.2)	Opportunistic capture	When a new session's similarity search returns a past match at or above the confirm-at-build-time threshold (§22) with no recorded outcome, the outcome prompt is shown before the new session begins, and is skippable in one tap.	Must
FR-28 (new, v1.2)	Structured status required	A session_outcomes row always has a non-null status from the fixed set of three; narrative is never required to save an outcome.	Must
FR-29 (new, v1.2)	Judge outcome-awareness	When a past_context entry has a recorded outcome, the Judge's payload includes status and, if present, narrative_summary; a diff of the v1.2 Judge prompt against v1.1's shows only the additions described in §20.7.	Must

ID	Requirement	Acceptance criterion	Pri.
FR-30 (new, v1.2)	Manual outcome entry	An outcome can be added or edited from a past session's detail view at any time, independent of whether the opportunistic prompt (FR-27) ever fired for it.	Should
FR-31 (new, v1.2)	Export completeness	The export capability (FR-25) includes any recorded session_outcomes row for each exported session, with narrative encrypted-by-default under the same posture as raw_narrative.	Must

13. Non-Functional Requirements

- Security — per Section 8; no additional NFRs beyond what is already scoped there.
- Reliability — Supabase's free-tier auto-pause after 7 days of inactivity is an expected, handled condition (a "reconnecting" state), not a defect — the same posture Phase 1 applied to Streamlit's cold-start (v0.2.2 FR-8). Web Speech API's variable browser support is an expected condition handled by FR-14's fallback, not a defect.
- Compliance — per Section 8; still not applicable at single-user scale, though the advisory-disclaimer language in §19 is strengthened now that real content persists.
- Performance / cost ceiling — $0 hard ceiling, tightened definition per §0 (no depleting-credit products count as zero-cost).

14. Testing & Quality Gates

Priority unit / integration test targets

- Voice transcription accuracy spot-checked across at least two browsers/devices (e.g. Chrome desktop and one mobile browser).
- RLS policy verified using only the anon key — confirms FR-16 directly, not just by code review.
- Encryption round-trip verified — confirms FR-17 directly against the live Supabase table, not just against local test fixtures.
- Reasoning-engine parity diffed line-by-line against v0.2.2 §20 — confirms FR-18.
- Supabase auto-pause/wake behavior tested deliberately where practical. Note: genuinely waiting out a 7-day inactivity window before every release isn't practical for solo QA — use Supabase's manual pause control if available as a stand-in, and treat full pre-launch coverage of this path as an honestly-documented gap (§15), not a false claim of completeness.
- Opportunistic outcome prompt tested against a deliberately re-run scenario (§21.3) — confirms FR-27 fires at the intended similarity threshold, not just that the query executes.
- Judge outcome-weighting spot-checked: run the same scenario twice with a status of deviated and a clearly negative narrative_summary recorded after the first run, and confirm the second run's pattern_note actually references it — confirms FR-29 does something, not just that the field exists.

Security-specific gates

- Secret scan (gitleaks) clean before every commit, now checking two secret patterns.
- The same three Antigravity verification tests from v0.2.2 §14, re-run against the new Next.js codebase rather than assumed to still hold because they passed once for Streamlit: auto-run refusal, secret-exposure refusal (now covering both keys), and prompt-injection resistance.
- session_outcomes.narrative encryption round-trip verified with the same rigor as raw_narrative — confirms FR-26, not assumed identical by similarity to FR-17.

Manual QA checklist before any demo or release

☐ Full session end-to-end on desktop browser, by typing.

☐ Full session end-to-end on phone browser, by voice.

☐ Supabase cold-"waking" scenario tested (project left idle before demo, or manually paused).

☐ No API key or database credential visible anywhere in the repo, chat history, or browser devtools — both secrets checked, not just one.

☐ Content-hygiene reminder visible before first submission.

☐ At least one session deliberately inspected in the Supabase table editor for schema conformance and to confirm raw_narrative is genuinely unreadable as ciphertext.

☐ Scenario A (v0.2.2 §21.3) run end-to-end by voice at least once.

☐ Scenario B (v0.2.2 §21.4) run end-to-end; Strategist's domain_framing still correctly switches to "adversarial."

☐ Session delete verified to actually remove the Supabase row, not just hide it client-side.

☐ Opportunistic outcome prompt triggered at least once, answered with each of the three status options across separate test runs, and skipped at least once — confirming none of the four paths breaks the new-session flow.

☐ Manual outcome entry/edit exercised from the case-history detail view, independent of the opportunistic prompt.

☐ session_outcomes.narrative confirmed unreadable as ciphertext via direct table inspection, same check already performed for raw_narrative.

15. Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
Gemini Live API creeps back in mid-build as a scope-creep temptation	Medium	Medium	Design-freeze rule (§6) plus this document explicitly naming it Won't-Have (§5).
Supabase free-tier auto-pause causes a bad first-open experience	Medium	Low	FR-handling with an explicit "waking" state, same UX pattern as Phase 1's cold-start.
Web Speech API browser inconsistency, notably in	Medium	Medium	Typed fallback (FR-14), tested on at least two engines before release.

Risk	Likelihood	Impact	Mitigation
some non-Chromium mobile browsers			
Encryption key loss — if the Vercel env var holding the symmetric key is ever lost or rotated without a migration plan, all past narrative content becomes permanently unreadable	Low	High	Back up the key outside Vercel in at least one secure location before storing any real narrative content. This is a real operational risk for a solo developer and is named plainly rather than assumed away — see §22 open item.
Hard cutover leaves no fallback if Phase 2 has a launch-week defect	Medium	Medium	Explicitly accepted per the founder's own migration decision (§21.4); mitigated by thorough M6 testing before cutover, not by a parallel run.
Vercel or Supabase account-level compromise exposes unencrypted fields (persona/Judge JSON, objective/constraint text)	Low	High	Accepted risk, disclosed in §8 rather than hidden — narrower in scope than encryption's coverage, but broader than Phase 1's public-repo-only risk.
Outcome data develops a self-report / survivorship skew (new, v1.2) — people are more likely to log outcomes for decisions that clearly resolved, one way or another, than ones that faded out ambiguously, biasing the recorded pattern over time	Medium	Medium	The opportunistic prompt (FR-27) is symmetric — it fires on similarity, not on anticipated sentiment. The Judge prompt (§20.7) is explicitly instructed not to over-index on a small or one-sided outcome sample. Named here rather than assumed away, since it's an epistemic risk, not a technical one — no technical fix fully closes it.
Opportunistic prompt becomes an annoyance if the similarity threshold is poorly tuned (new, v1.2) — either firing on loosely-related sessions or almost never firing	Medium	Low	Threshold is explicitly a confirm-at-build-time value (§22), not hardcoded as final; always skippable in one tap (FR-27); manual entry (FR-30) remains available regardless of how the threshold is tuned.

16. Repository & Environment Hygiene

- Branching — main plus one feature branch per milestone; no direct commits to main, including by the Product Owner. Unchanged from v0.2.2.
- Merge rules — human review required for every merge; no auto-merge, ever, including agent-authored diffs. Unchanged.

- Environments — a single environment: Vercel production is production, same accepted trade-off framing v0.2.2 used for Streamlit Community Cloud. Local development uses `next dev` against the same free-tier Supabase project, used carefully rather than against a separate paid staging instance.
- Secrets now cover two keys, both as Vercel environment variables — the Gemini API key and the Supabase service-role key — neither ever in the repo, neither ever in a local .env file that gets committed.

17. Success Metrics & Definition of Done

Engineering metrics

- Zero-cost adherence verified at each milestone across all three providers (Vercel, Supabase, Google AI Studio), not just the Gemini API as in Phase 1.
- 100% of Section 14 security and schema-validation gates pass before Phase 2 sign-off.
- Defect tracking remains informal/manual at this scale — no CI dashboard is warranted.

Product / outcome metrics

Tied to Section 3's proposed Phase 2 success definition: voice-first sessions work end-to-end, at least one past decision's case history is genuinely retrievable in a later session, and the $0 ceiling held throughout.

Definition of Done

☐ All Functional Requirements FR-12 through FR-21 (Section 12) pass their acceptance criteria, and FR-1 through FR-11 still pass against the Next.js implementation.

☐ All Section 14 security and schema-validation gates pass, including the three re-run Antigravity verification tests.

☐ App confirmed usable end-to-end on both desktop and phone browsers, by both voice and typing.

☐ $0 spend confirmed across every provider in the stack — Vercel, Supabase, and Google AI Studio.

☐ The Streamlit app is retired per the §21.4 runbook.

☐ This PRD saved into the project's frozen-spec location for any future Antigravity session to reference.

18. Future Features / Post-Launch Roadmap

Integration with the separate personal knowledge-hub project — deferred, not rejected

The founder is separately planning a broader personal knowledge hub (Perplexity, Gemini, Claude, NotebookLM, and Obsidian, working together as memory plus agent tooling). As of this writing that project is an idea with no architecture decided — no data model, no chosen store of record, no defined interface. O.D.I.N.'s recall feature (§5, §10, §20.7) is deliberately built standalone rather than integrated against that project now, for the same reason the Gemini Live API and Chroma Cloud were rejected in §0: building against an unspecified dependency risks rework and breaks the self-imposed timeline. This is a deferral, not a decision that the two should stay separate forever — O.D.I.N.'s decision histories are exactly the kind of structured personal knowledge that belongs in a broader hub eventually.

What makes the deferral safe rather than a dead end: O.D.I.N.'s memory sits on a deliberately boring, portable substrate — plain Postgres, a documented schema, an explicit JSON export (FR-25, FR-31), and the embedding model versioned alongside every vector (§10). When the knowledge-hub project has its own defined architecture, integrating O.D.I.N. becomes a scoped, separate project consuming that export — not a rebuild of O.D.I.N.'s memory feature. Revisit this once the other project has a PRD of its own.

Outcome-aware personas — deferred, distinct from the recall deferral above

v1.2 keeps outcome data Judge-only, for the same reason v1.1 kept past syntheses Judge-only: the Quant, Strategist, and Behaviorist reason independently by design, and feeding any of them prior context risks quietly collapsing that independence. Outcome data is a genuinely different case from past reasoning, though — it's validated ground truth ("this path was taken and it went badly"), not another opinion competing with the persona's own. That's a real argument for eventually letting at least the Behaviorist factor in aggregate outcome patterns (e.g. "in similar situations, following this kind of recommendation has gone poorly N times"). It's deliberately not built here — it deserves its own explicit design decision and its own PRD version, the same discipline that governed the Judge's amendment in §20.7, not a quiet extension riding in on this addendum.

Other post-launch items

- Multi-user authentication (Supabase Auth) — only if O.D.I.N. is ever opened to more than one person; RLS is already in place to build on.
- Distinct synthesized voices per persona — cosmetic, low priority.
- LangGraph-based autonomous agent state machine with a Python interpreter tool for live Monte Carlo / linear-programming runs (Phase 3) — unchanged from v0.2.2.
- Any service-mesh identity, ML-driven DLP, formal Policy-as-Code / GRC auditing, AI SBOMs — only if O.D.I.N. is ever productized at enterprise scale, unchanged.
- BASIC/A2AS agent-security framework and subagent swarm topology — only if Phase 3's autonomy genuinely warrants it, unchanged.

19. Other Important Matters

- Advisory disclaimer — carried over and strengthened: O.D.I.N. is a decision-support reasoning tool, not a substitute for licensed professional advice. Now that narrative content persists indefinitely by default, the owner is responsible for managing what's stored — including using the session-delete capability (FR-21, pending confirmation per §22) if desired.
- Binding terminology — carried over: always "the Judge," never "arbitrator" or "referee." "Streamlit Community Cloud" now refers only to the retired Phase 1 infrastructure — do not describe it as currently live once cutover (§21.4) completes. The repo remains public by deliberate choice; persona and Judge outputs remain JSON internally, with Markdown/Mermaid reserved for the rendered, presentation-only report — don't conflate the two.
- Fallback for the soft 4–6 week deadline — unchanged in spirit from Phase 1: if the target slips, the roadmap simply extends; there is no external stakeholder or launch event to notify.

20. Persona Prompt Specification & Output Schema

The Quant, Strategist, and Behaviorist specifications (§20.4–20.6) are carried over unmodified from PRD v0.2.2, per the design-freeze rule and Guiding Priority #3 — byte-for-byte, confirmed by FR-18. The Judge specification (§20.7) now carries two deliberate, versioned amendments: v1.1 added the optional past_context input and pattern_note output; v1.2 enriches past_context with outcome data and adds explicit instruction on how to weigh it. Neither is a casual mid-build edit — each is the new PRD version the design-freeze rule itself requires before any prompt change touches code. Everything else in this section is reproduced in full so this document remains a standalone single source of truth.

20.1 Field routing rule

All three intake fields — core_objectives, known_constraints, raw_narrative — are passed identically and in full to the Quant, Strategist, and Behaviorist calls. No field is withheld from any persona. The Judge call does not receive the raw intake fields again; it receives only the three completed persona JSON envelopes, verbatim.

20.2 Output format requirement

All four calls use Gemini's structured-output mode (response_mime_type: "application/json" plus a responseSchema — confirm exact current parameter names in the Gemini API docs at build time). This is a hard requirement, not a preference: a prompt instruction alone ("please respond in JSON") is not sufficient and is known to fail often enough to be unreliable.

20.3 Common persona output envelope

Every one of the three persona calls returns this envelope, extended with persona-specific fields defined in 20.4–20.6:

```
{
  "persona": "quant" | "strategist" | "behaviorist",
  "summary": string,       // 1-2 sentence headline
  "reasoning": string,     // full analysis, markdown-flavored text
  "key_points": [string],  // 2-5 extracted takeaways
  "confidence": "low" | "medium" | "high"
}
```

20.4 The Quant — Operations Research & Statistics

Schema extension:

```
{
  ...envelope,
  "paths": [
    {
      "name": string,
      "probability": number | null,  // null when unsupported by input
      "expected_value_notes": string
    }, ...
  ]
}
```

System prompt:

```
You are the Quant, one of four independent reasoning engines
inside O.D.I.N. Strip out emotional variables and narrative
framing. Treat the input as a cold optimization problem.
```

```
For each realistic path forward, estimate expected value.
Give a numeric probability ONLY where the narrative contains
enough concrete signal to support it non-arbitrarily -- observed
behavior patterns, response times, stated timelines, or other
quantifiable constraints. Where the input does not support a
number, set probability to null and reason qualitatively about
relative likelihood instead. Never invent false precision.

Identify the mathematically strongest path and at least one
credible alternative. Weigh both best-case and worst-case
outcomes for each path, not just the hoped-for outcome.

Return ONLY valid JSON matching the provided schema. No prose,
no markdown fences, outside the JSON object.
```

20.5 The Strategist — Game Theory & Sun Tzu

Schema extension:

```
{
  ...envelope,
  "domain_framing": "adversarial" | "non_adversarial",
  "reversibility_ranking": [
    {
      "move": string,
      "reversibility": "low" | "medium" | "high",
      "notes": string
    }, ...
  ]
}
```

System prompt:

```
You are the Strategist, one of four independent reasoning
engines inside O.D.I.N. First determine, from the input, whether
this is a competitive domain (rivals, negotiation, business
competitors) or a personal, non-adversarial domain. State this
in domain_framing.

In a competitive domain: model counter-moves, payoff matrices,
and equilibria in the classic adversarial sense.

In a non-adversarial domain: apply the same toolkit without
casting anyone as an opponent. Map timing, signal-reading, and
reversibility -- which moves can be walked back and which can't
-- and rank them from lowest to highest reversibility risk in
reversibility_ranking. Recommend sequencing the lowest-risk move
first rather than jumping to the highest-stakes one.

In either domain, look for a stable equilibrium and note where a
classic stratagem applies -- asymmetric advantage, patience,
reshaping the situation rather than forcing a premature move.

Return ONLY valid JSON matching the provided schema. No prose,
no markdown fences, outside the JSON object.
```

20.6 The Behaviorist — Behavioral Economics & Risk Auditing

Schema extension:

```
{
  ...envelope,
  "biases_detected": [
    {
      "bias": string,       // e.g. "sunk_cost_fallacy"
      "present": boolean,
      "evidence": string    // quote/paraphrase from narrative,
                            // or "not observed" if present=false
    }, ...
  ]
}
```

System prompt:

```
You are the Behaviorist, one of four independent reasoning
engines inside O.D.I.N. -- O.D.I.N.'s psychological fail-safe.
Scan the user's own narrative for blind spots.

At minimum, explicitly evaluate three biases every time, even
when absent: sunk_cost_fallacy, confirmation_bias, and
overconfidence. Report each with present=true/false and cite the
specific piece of the narrative that supports or rules it out --
never assert a bias without textual evidence, and never assert
one that is not actually supported by the input.

Beyond that floor, add any other bias or distortion you detect
(e.g. fear of rejection dressed up as patience, loss aversion,
anchoring, social-desirability framing) as additional entries in
biases_detected, with the same evidence standard.

Flag when the decision as narrated looks driven by ego or
fatigue rather than the facts on the table -- say so plainly in
the reasoning field.

Return ONLY valid JSON matching the provided schema. No prose,
no markdown fences, outside the JSON object.
```

20.7 The Judge — First Principles Arbitration (amended in v1.1, extended in v1.2)

Input, amended: the Judge still receives the three completed persona envelopes verbatim, unchanged since v0.2.2. New in v1.1, it additionally receives an optional past_context array, populated by the retrieval flow in §10, empty on a person's first few sessions. New in v1.2, each entry may also carry outcome data when one was recorded:

```
past_context: [
  {
    date: string,
    synthesis: string,
    outcome: {                    // new in v1.2 -- null if
      status: "followed_path"     // no outcome was ever
            | "deviated"          // recorded for that
            | "still_deciding",   // past session
      narrative_summary: string | null
    } | null
  },  // 0-3 entries, most similar past sessions' Judge
      // syntheses (and outcomes, if recorded) only
  ...
]
```

Output schema, amended (one new optional field, added in v1.1; unchanged in v1.2):

```
{
  "synthesis": string,
  "tension_points": [string],       // where personas collided,
                                     // and how it was resolved
  "recommended_path": string,
  "next_3_actions": [string, string, string],  // exactly 3,
                                                 // sequenced
  "mermaid_diagram": string,         // valid Mermaid.js syntax
  "pattern_note": string | null      // added in v1.1
}
```

System prompt, amended (v1.1 and v1.2 additions both shown inline, not hidden):

```
You are the Judge, the First Principles Arbitrator -- the fourth
and final call inside O.D.I.N. You receive three completed,
independent JSON outputs from the Quant, the Strategist, and the
Behaviorist, all reasoning over the same decision. Do not
re-derive their analysis -- synthesize it.

Identify where their conclusions agree, and record every place
they collide in tension_points -- for example, where the Quant's
highest-expected-value path conflicts with a risk the Strategist
or Behaviorist flagged. Where they collide, strip the problem to
its physical and logical first principles and weigh survival and
irreversibility over short-term optimization.

Produce exactly three concrete, sequenced next actions in
next_3_actions -- not four, not two, not vague restatements of
the recommended_path -- ordered by what should happen first.

Include a valid Mermaid.js flowchart in mermaid_diagram
representing the decision path and its key branch point(s).

[ADDED IN v1.1] You may also receive past_context: 0-3 syntheses
from the person's earlier decisions, most similar to this one.
Treat past_context strictly as supplementary pattern-recognition
input, never as authoritative and never as a substitute for this
session's independent reasoning. If a genuine recurring pattern
is visible -- the same hesitation, the same kind of tradeoff, a
repeated outcome -- name it specifically in pattern_note and, if
relevant, in synthesis or tension_points. If past_context is
empty, or nothing meaningful connects, set pattern_note to null.
Never fabricate a pattern to fill the field.

[ADDED IN v1.2] Some past_context entries may include an outcome:
what actually happened after that earlier decision. Weigh a
recorded outcome more heavily than a synthesis with no outcome --
a validated result is stronger evidence than an untested
conclusion. If status is followed_path and the outcome was
clearly poor per narrative_summary, that is a meaningful signal
worth naming plainly, even if it complicates the current
recommendation. If status is deviated and things went well anyway,
note that too -- it may mean the earlier reasoning was overly
cautious. Do not treat outcome: null, or the absence of past_
context entirely, as a negative signal of any kind -- it usually
just means too little history exists yet. Do not generalize from
a single past outcome as if it were a proven pattern; say so
explicitly if the evidence is thin.
```

```
Return ONLY valid JSON matching the provided schema. No prose,
no markdown fences, outside the JSON object.
```

20.8 Implementation note for Antigravity

The Quant, Strategist, and Behaviorist prompts and schemas are frozen exactly as they were in v0.2.2 — the design-freeze rule requires no change, and none was made across v1.1 or v1.2. The Judge's two amendments above are the only deliberate exceptions, each made through its own versioned document rather than mid-build, per §6. Phase 2 also changes how all four calls are hosted (Next.js API routes instead of Streamlit's backend) and adds the persistence, retrieval, and now outcome-capture steps around the Judge call — none of it touches the Quant, Strategist, or Behaviorist prompts.

21. Voice Interaction, UI Copy & Migration Runbook

21.1 Purpose

This section carries forward v0.2.2 §21's role — the exact copy strings Antigravity should display verbatim — and extends it with the two things unique to a phase-transition PRD: voice-specific status copy, and a concrete runbook for the hard cutover decided in §0.

21.2 Progressive-disclosure status strings

The four original strings are unchanged — the underlying four calls didn't change, only their host did. Display verbatim; do not paraphrase:

- **Quant call in flight:** "Running Quantitative Engine…"
- **Strategist call in flight:** "Applying Game Theory Payoff Matrix…"
- **Behaviorist call in flight:** "Auditing for Cognitive Bias…"
- **Judge call in flight:** "Arbitrating Final Blueprint…"

New strings, specific to Phase 2's voice and persistence layers:

- **Microphone active:** "Listening…"
- **Web Speech API processing:** "Transcribing…"
- **Supabase reconnecting after auto-pause:** "Waking the archive…"

New in v1.2, the opportunistic outcome prompt. Shown before a new session begins, only when a similar past session has no recorded outcome. Copy pattern, not a single fixed string, since it references the past session's date:

- **Prompt header:** "You had a similar decision on [date] — how did it go?"
- **Status buttons (single tap, one required to proceed past this step):** "Followed the plan" · "Went a different way" · "Still deciding"
- **Optional detail field placeholder:** "Add a note (optional)"
- **Always-available escape:** "Skip" — dismisses without recording anything; does not count as still_deciding, and does not block starting the new session.

21.3 Scenarios A & B — carried over as QA fixtures

Both worked scenarios from v0.2.2 §21.3–21.4 remain valid, unmodified test fixtures — the underlying reasoning engine didn't change. In Phase 2, both are additionally run by voice at least once each as part of Section 14's manual QA checklist, to confirm the Web Speech API transcription path doesn't degrade input quality compared to typing. As of v1.2, running either scenario twice — with an outcome recorded after the first run — is the recommended way to manually verify the opportunistic prompt and outcome-aware Judge behavior end-to-end. Refer to v0.2.2 §21.3–21.4 for the full text of each scenario; it is not reproduced a second time here to avoid the two documents drifting out of sync on wording that didn't actually change.

21.4 Migration runbook — hard cutover

Per the founder's explicit decision (§0): no parallel run. The Streamlit app is retired the moment Phase 2 build starts, not once Phase 2 ships. The sequence:

- 1. Tag the current Streamlit codebase (e.g. git tag phase-1-archive) so it remains recoverable as a historical reference, even though it will no longer run live.
- 2. Build Phase 2 fully through Milestone 6, including the full Section 14 QA pass — there is no live Phase 1 fallback to lean on if Phase 2 has a defect, so this pass carries more weight than Phase 1's own equivalent did.
- 3. Undeploy the Streamlit Community Cloud app once Milestone 6 passes — do not leave it publicly live alongside Phase 2 (FR-20).
- 4. Update any personal bookmarks or shortcuts to the new Vercel URL.
- 5. From this point forward, Phase 2 is the only live O.D.I.N. instance; “Streamlit Community Cloud” in any future document refers only to retired infrastructure (§19).

22. Appendix

Traceability

PRD section	Primary source(s)
Sections 0–4, 6, 9, 11–19, 21.1–21.2, 22	This document's founder interview, 12 Aug 2026 (recorded in full across this conversation's turns)
Section 5 (MoSCoW)	Founder interview, extended with one proposed Should-Have (session delete) flagged for confirmation
Sections 7, 8, 10 (tech stack, security, data model)	Founder interview plus live verification of Gemini Live API and Chroma Cloud pricing terms, 12 Aug 2026
Section 20.4–20.6 (Quant/Strategist/Behaviorist)	PRD v0.2.2 §20, reproduced verbatim per the design-freeze rule
Section 20.7 (The Judge)	PRD v0.2.2 §20.7 plus a versioned v1.1 amendment (past_context / pattern_note), per this document's own review
Section 21.3 (Scenarios A & B)	PRD v0.2.2 §21.3–21.4, referenced rather than reproduced to avoid drift

PRD section	Primary source(s)
Section 21.4 (Migration runbook)	Founder interview — hard-cutover decision, new to this document
Sections 5, 10, 12, 18, 20.7 (v1.1 recall & portability additions)	This document's own review, 12 Aug 2026 — the recall feature and the personal-knowledge-hub integration question
Sections 5, 8, 10–12, 15, 18, 20.7, 21.2 (v1.2 outcome-feedback additions)	This document's own review, 13 Aug 2026 — the outcome-tracking gap and its resolution

Glossary

Carried over from v0.2.2 where still relevant (RACI, MoSCoW, HITL, MCP, cold-start, free-tier quota, structured output / responseSchema, envelope), plus terms new to Phase 2:

- **RLS (Row-Level Security):** a Postgres feature restricting which rows a given database role can read or write, enforced at the database layer rather than only in application code.
- **pgvector:** a Postgres extension adding a vector data type and similarity search, used here for semantic recall over case history.
- **Web Speech API:** a browser-native JavaScript API providing speech recognition (SpeechRecognition) and speech synthesis (SpeechSynthesis), with no server round-trip and no per-use cost.
- **Serverless (vs. persistent-process):** a hosting model where code runs only in response to a request rather than as an always-on process; Streamlit Community Cloud used the latter, Vercel's API routes use the former.
- **Starter credit:** a one-time free usage allowance that converts to metered billing once exhausted — distinct from a durable free tier, and the specific distinction that disqualified Chroma Cloud under §0's tightened zero-cost rule.
- **Opportunistic capture (new in v1.2):** prompting for feedback at the moment it's naturally relevant — here, when a new session closely resembles a past one — rather than via a scheduled reminder or a passive, never-visited log.
- **Outcome (new in v1.2):** a recorded status (followed_path / deviated / still_deciding) and optional narrative describing what actually happened after a decision, distinct from what the Judge concluded at the time.

Open questions

This document is honest about what remains genuinely open rather than claiming false completeness:

- **1. Session delete/purge capability (FR-21):** added as a Should-Have based on reasonable inference, not explicitly confirmed in the interview. Confirm before Milestone 3, since it affects the Supabase schema.
- **2. Encryption-key backup/recovery process:** named as a real operational risk (§15) without a fully specified answer. Decide on a concrete backup location and process before Milestone 3, before any real narrative content is stored. As of v1.2, this same key also protects session_outcomes.narrative, raising the stakes on getting it right.

- **3. Gemini 3.5 Flash's exact free-tier RPD/RPM/TPM limits:** confirm in Google AI Studio at build time — the same confirm-at-build-time posture v0.2.2 already applied to its own model string.
- **4. Current Web Speech API browser support matrix:** changes over time; confirm fresh at build time rather than relying on this document's writing-date assumptions.
- **5. (Resolved in v1.1, kept here for the record) Whether O.D.I.N.'s memory should integrate with the separate personal knowledge-hub project:** answered — not now, deferred until that project has its own architecture (§18). Revisit when that project has a PRD; nothing further to decide here until then.
- **6. (New in v1.2) Similarity threshold for the opportunistic prompt:** not yet tuned against real data — propose a starting cosine-similarity threshold of 0.75 and adjust based on whether N2's manual testing shows it firing too often, too rarely, or on genuinely unrelated sessions. Confirm and record the final value at build time, same posture already used for the model string and browser-support matrix above.

Source documents

- 260808 — O.D.I.N. PRD v0.2.2 (frozen Phase 1 spec; this document builds on it, does not replace it)
- 260812 — Phase 2 founder interview (v1.0/v1.1 gap-closure record)
- 260813 — Outcome-feedback review (v1.2 gap-closure record, this document)
- All protocol documents traced in v0.2.2 §22 — still governing, unchanged by this pivot